HUMAN EFFORT, DELIVERY SPEED AND RETAINED OUTPUT IN AI-ASSISTED SOFTWARE ENGINEERING

Abstract. A code-change index does not, by itself, measure human labour or delivery speed. This distinction becomes consequential when AI assistance changes the production process, the allocation of review work and the observation of defects. We formulate a task-level accounting model that separates all-role person-hours, elapsed time to acceptance and output retained after a stated follow-up. An explicit observational-equivalence construction shows why commit events cannot guarantee identification of human activity without additional restrictions or observations. Elementary parallel-execution, retry and defect-detection models then establish separate conditions for effort and speed improvements, including cases in which recorded throughput rises while accepted correctness falls. The resulting contribution is a bounded measurement specification and a feasible empirical study design, rather than a new probability theory or an estimated AI effect. Targeted corrections to the Standard Coder argument, reproducible synthetic examples and a proof-coverage concordance make the analytical claims inspectable. An executable Rust extension evaluates repeated cohort-horizon queries and exposes the trade-off between indexing cost and repeated scans. No company data are used, and empirical calibration, causal identification in a particular organisation and practical superiority remain open.

1. The measurement question

Suppose an engineering team delivers the same requested change after adopting an AI coding assistant. Its commits may arrive sooner, contain more code, require less typing or need more review. None of these observations alone tells us how many person-hours the team supplied, when acceptable functionality became available or whether that functionality remained acceptable. The question addressed here is what must be observed and held fixed to distinguish those outcomes.

Wright and Ziegler's Standard Coder approach [12] provides a useful starting point. It infers coding activity from commit timing and then predicts the resulting labels from code-change features. The output, Standard Coding Hours (SCH), can serve as a common reference index. Its interpretation nevertheless depends on two links: the link from events to human activity, and the link from a code representation to a stable unit of work. AI can alter either link. A reference score may remain useful for describing changes while ceasing to track current labour.

The case for several measures is established in software-engineering research. The SPACE framework, for example, places activity and efficiency within a broader account of developer productivity [6]. Mozannar et al. develop the CUPS activity taxonomy and examine interaction time costs in AI-assisted programming [9]. The present article addresses a narrower accounting and identification problem: how to define the labour, timing and retained-output quantities needed for a defensible comparison when production and verification can move between people and agents. It does not attempt to reduce satisfaction, cognitive strain, organisational value or all aspects of software quality to time.

Existing AI studies also illustrate why the endpoint matters. Peng et al. compare completion time among completers of a specified programming exercise [10]. Becker et al. study experienced maintainers in familiar repositories, reporting a forecast-adjusted contrast from a log-time model [1]. Cui et al. study weekly completed pull requests in three organisational experiments [5]; their June 2025 manuscript uses random assignment as an

Date: Methodological draft with performance extension, 29 October 2026.

Key words and phrases. software engineering, human effort, AI assistance, measurement, identification, retained output.

instrument for adoption in a weighted instrumental-variable analysis [4, Section 4.1 and Table 3]. Their interventions, populations and outcome definitions differ. Their estimates therefore cannot supply interchangeable parameters for a universal productivity equation. METR's subsequent account of selection and concurrent-agent measurement difficulties gives a further reason to record work and elapsed time separately [2]. These studies motivate the question; their effects are not re-estimated here.

The argument has three parts. First, an observation model exposes what event telemetry cannot identify without an external anchor. Secondly, task accounting fixes the units and observation horizons before a small execution model explains how effort and speed can diverge. Thirdly, a proposed study measures those endpoints under a common acceptance and follow-up procedure. The mathematical ingredients—finite probability, conditional expectation, sampling weights, dependency paths and renewal rewards—are standard. The practical contribution is their explicit alignment with the measurement problem, including failure cases that prevent an index from being mistaken for an observed outcome. No claim is made that this synthesis is an exhaustive literature review or a validated forecasting system. Section 7 adds a bounded implementation experiment: speeding up repeated accounting queries while preserving their integer totals. Its machine-runtime measurements do not measure human productivity or an AI policy effect.

2. An event history is an observation of work

Let a minute-indexed hidden variable $Z_t \in \{0,1\}$ indicate an activity state, and let $C_t \in \{0,1\}$ record a commit event. A fitted event model specifies both activity dynamics and an emission mechanism. Calling $Z_t = 1$ "coding" supplies a semantic interpretation, but it does not establish that the state equals independently measured labour. Moreover, the same event law can arise from different activity laws.

Proposition 2.1 (Failure of unrestricted identification)**.** *Suppose the states Z_t are independent Bernoulli variables with parameter $\pi \in (0,1)$, and, independently across minutes and of the states, B_t are Bernoulli variables with parameter $c \in (0,1)$. Define $C_t = Z_t B_t$. The complete law of the event process depends on (π, c) only through πc. In particular, $(\pi, c) = (1/2, 1/5)$ and $(1/4, 2/5)$ give identical observed laws but different expected active times.*

Proof. Each C_t is Bernoulli with parameter $\theta = \pi c$, and the C_t are independent. For every $n \geq 1$ and every binary string $(e_1, \ldots, e_n)$,

$$\mathbb{P}(C_1 = e_1, \ldots, C_n = e_n) = \theta^{\sum_t e_t}(1-\theta)^{n-\sum_t e_t}. \tag{1}$$

Both parameter pairs give $\theta = 1/10$. In contrast,

$$\mathbb{E}\left[\sum_{t=1}^{n} Z_t\right] = n\pi, \tag{2}$$

which is $n/2$ minutes in the first model and $n/4$ in the second. Thus the same observable distribution corresponds to two different target values. □

This construction belongs to the independent-state subfamily of a two-state hidden Markov model: both transition rows give the same next-state probability. It disproves identification throughout an unrestricted family, not identification in every persistent or otherwise restricted model. Additional restrictions, informative code marks or independent activity observations may distinguish the relevant cases. A guarantee must state which of those restrictions or observations supplies the information. AI makes the question more immediate because commit production can continue while human attention is elsewhere.

Internal agreement between two estimates of the state cannot replace that information.

Proposition 2.2 (A consistency identity and its limit)**.** *For nested information sets $\mathcal{F} \subseteq \mathcal{G}$ and an integrable random variable Z, let $p = \mathbb{E}[Z \mid \mathcal{F}]$ and $q = \mathbb{E}[Z \mid \mathcal{G}]$ under the true joint law. Then*

$$\mathbb{E}[q - p \mid \mathcal{F}] = 0. \tag{3}$$

The identity does not imply that a pair of proposed estimates equals those conditional expectations.

Proof. The tower property gives $\mathbb{E}[q \mid \mathcal{F}] = p$. For the converse, a procedure returning the same constant $p = q = b$ has zero correction for any underlying activity process, including one whose conditional activity probability differs from b. □

Such a diagnostic can find incompatibilities within a model, but passing it does not establish activity calibration. Parameters used in a prospective prediction must also be fitted without future observations; restricting the prediction inputs while fitting parameters to future events is still retrospective. Appendix A gives targeted arithmetic and inferential corrections to the Standard Coder paper. The present measurement specification does not depend on reproducing its unavailable empirical data.

3. Task accounting with explicit boundaries

3.1. **The task, the policy and the person-hour.** Enrol task i at time s_i, when its requested behaviour, eligibility and acceptance criteria are recorded. Task identity follows the requirement, rather than a commit or pull request. Before observing outcomes, define how split tasks, merged tasks and scope changes will be handled. A parent requirement can retain its original identity while children record implementation activity; adding the parent and its children as separate delivered output would count the same work twice. Material changes of scope require a versioned record and a stated analysis rule.

An *AI policy* specifies available tools and model versions, permitted autonomy, eligible tasks, review responsibilities and release rules. Assignment to such a policy is different from actual use. Difficult tasks may elicit more use, and people may decline an assigned tool. The assigned policy is the intervention in the proposed experiment; actual exposure remains a recorded behaviour.

Let $a_{ijr}(t)$ be the fraction of person j's time allocated to task i in role r at time t. For a finite set of observed people, measurable allocation functions satisfy

$$a_{ijr}(t) \geq 0, \qquad \sum_{i,r} a_{ijr}(t) \leq 1, \qquad H_i(u) = \sum_{j,r} \int_{s_i}^{s_i+u} a_{ijr}(t)\, dt. \tag{4}$$

With time in hours, $H_i(u)$ has units of person-hours. Roles include specification, implementation, prompting, review, testing, integration and repair. Pair work can contribute two person-hours in one elapsed hour. Conversely, a person's two concurrent sessions cannot each receive a full hour of that person's attention. State whether attended waiting belongs in the labour total; unattended execution is recorded as agent resource use.

Equation (4) defines the quantity to be measured. It does not assert that allocations are known precisely. Shared review and platform work need an attribution rule, including an overhead category when task allocation is indefensible. Record agent instance-hours by resource class, tokens and charges separately. Adding them to person-hours requires an explicit economic conversion and changes the question from labour to cost.

Let T_i be elapsed time from enrolment to first acceptance. If the task is permanently abandoned before acceptance, put $T_i = \infty$. Loss of observation is censoring, whereas abandonment is an outcome. Deployment to users is a separate event whenever acceptance and deployment differ. Starting the clock at the first commit would omit earlier reasoning and specification; starting it at enrolment includes queueing and makes that choice visible.

3.2. **A frozen reference and a follow-up assessment.** Assign a nonnegative reference score S_i^0 using a declared version and representation. It may count tasks, independent functional units or frozen SCH weights. A labour-calibrated reference would target

$$S^0(x) = \mathbb{E}[H^{\mathrm{ref}} \mid X = x] \tag{5}$$

for a specified reference population, resource boundary, quality bar and stopping rule, provided the expectation is finite. A predictor trained only on imputed coding labels targets those labels. Its calibration to independently measured labour requires a separate empirical test. The superscript 0 denotes a fixed reference, not current effort under the intervention.

For a fixed-scope causal comparison, requirements recorded at intake are preferable to final code differences: AI can change the latter. A retrospective code score can still be reported, with its dependence on the implementation made explicit. Splitting and squashing also matter because a nonlinear scoring rule need not be additive. Score a canonical task representation once, or use an additive requirement rubric. A failed attempt, retry or revert must not generate another unit of retained output.

Choose an acceptance horizon $\tau > 0$ and a follow-up duration $\Delta \geq 0$. For accepted tasks, let $R_i(\Delta) \in [0, 1]$ be the assessed fraction of the accepted requirement still meeting the rubric at time $s_i + T_i + \Delta$. Retained reference output is

$$Q_i(\tau, \Delta) = \begin{cases} S_i^0 R_i(\Delta), & T_i \leq \tau, \\ 0, & T_i > \tau. \end{cases} \tag{6}$$

The rubric must distinguish an intentional replacement from a defect-driven loss and explain fractional scores. Surviving lines, successful tests and absence of recorded incidents are observations of quality; none establishes complete correctness. Reliability of the assessment itself therefore belongs in the study.

For a finite intake cohort with positive total labour, define

$$P_H(\tau, \Delta) = \frac{\sum_i Q_i(\tau, \Delta)}{\sum_i H_i(\tau + \Delta)}. \tag{7}$$

The denominator includes labour on rejected, unfinished and abandoned tasks, and task-attributable work after acceptance within the fixed enrolment horizon. It is a cohort resource total. Because early acceptances have more of the horizon remaining, this boundary is not identical to stopping each task's repair accounting exactly Δ after acceptance. Either convention can be useful; they must not be substituted silently. Follow every enrolment for $\tau + \Delta$ to ascertain the numerator and this denominator.

A calendar-window rate answers another question. For a window W of positive length and a defined portfolio of tasks, let

$$V_W = \frac{\sum_{i:\, s_i + T_i \in W} S_i^0 R_i(\Delta)}{|W|}. \tag{8}$$

All counted acceptances need the same subsequent follow-up. Record work in progress at the window boundaries because some output may have begun earlier. Dividing portfolio output by the sum of overlapping task durations gives neither this calendar rate nor a team's capacity. Zero-effort tasks do not require individual output/effort ratios; a positive aggregate denominator and the count of such tasks are sufficient.

3.3. **An external anchor for labour.** Activity sampling offers a modest anchor without requiring continuous surveillance. Partition the observation period into finitely many person-intervals indexed by m. Let d_m be interval length, $a_m \in [0, 1]$ the target task's average allocation, and I_m indicate inclusion in a probability sample with known $\pi_m = \mathbb{E}[I_m] > 0$.

Proposition 3.1 (Design-unbiased labour accounting)**.** *Treat the allocations as fixed finite-population values. If sampled allocations are ascertained without error, then*

$$\widehat{H} = \sum_m \frac{I_m d_m a_m}{\pi_m}, \qquad \mathbb{E}_s[\widehat{H}] = \sum_m d_m a_m, \tag{9}$$

where $\mathbb{E}_s$ averages over the sampling design.

Proof. Linearity of expectation and $\mathbb{E}_s[I_m] = \pi_m$ give the equality term by term. Independence of inclusion indicators is unnecessary. □

This is the Horvitz–Thompson construction [8], not a new estimator. Its variance depends on the actual sampling design. In practice, short diaries, task records and an independently adjudicated subsample can estimate allocations and expose disagreement. Off-keyboard reasoning, review and coordination need explicit categories. Nonresponse, inaccurate retrospective labels and work outside the sampling frame are additional errors, not consequences removed by inverse-probability weights. Telemetry can be calibrated to the sampled observations before and after policy assignment. Missing AI-use records must remain missing rather than being coded as non-use.

4. Why effort and elapsed time can move independently

The accounting model does not require a particular execution mechanism. A small mechanism is nevertheless useful for deriving testable conditions and exposing invalid shortcuts. The models in this section describe specified executions; they do not fit an organisation.

4.1. Dependencies and parallel execution.

Represent the stages of one execution by a finite directed acyclic graph. Nodes have nonnegative elapsed durations; edges encode all required precedence constraints. Let C be its longest path duration. Let H and G denote total human person-hours and homogeneous-agent instance-hours required by these stages.

Proposition 4.1 (Resource and path bounds)**.** *With available capacities $m > 0$ people and $k > 0$ agents, every feasible schedule of elapsed length L satisfies*

$$L \geq \max\{C, H/m, G/k\}, \tag{10}$$

omitting a resource term when that class is absent. With unlimited resources and immediate execution of each ready node, the earliest completion time is C.

Proof. Every dependency path executes in order. Integrating resource capacity over the schedule gives $H \leq mL$ and $G \leq kL$. With unlimited resources, the earliest finish of a node is its duration plus the maximum predecessor finish time, with zero for an empty predecessor set. Induction along a topological ordering shows that the final finish is the maximum path duration. □

For finite resources, the maximum in (10) is only a lower bound. It does not solve the scheduling problem or describe queue interactions with other tasks. An agent-hour is comparable across nodes only within the declared resource class.

Consider one human production branch of duration $c_h \geq 0$ running independently beside one agent branch of duration $g \geq 0$, followed by human verification of duration $v \geq 0$. One attempt then consumes

$$h = c_h + v, \qquad t = \max(c_h, g) + v. \tag{11}$$

The maximum is justified by independent parallel branches. If a human stage needs the agent's result, its position in the graph changes and so does t. Figure 1 makes the person-time and elapsed-time distinction visible in a slow-agent example.

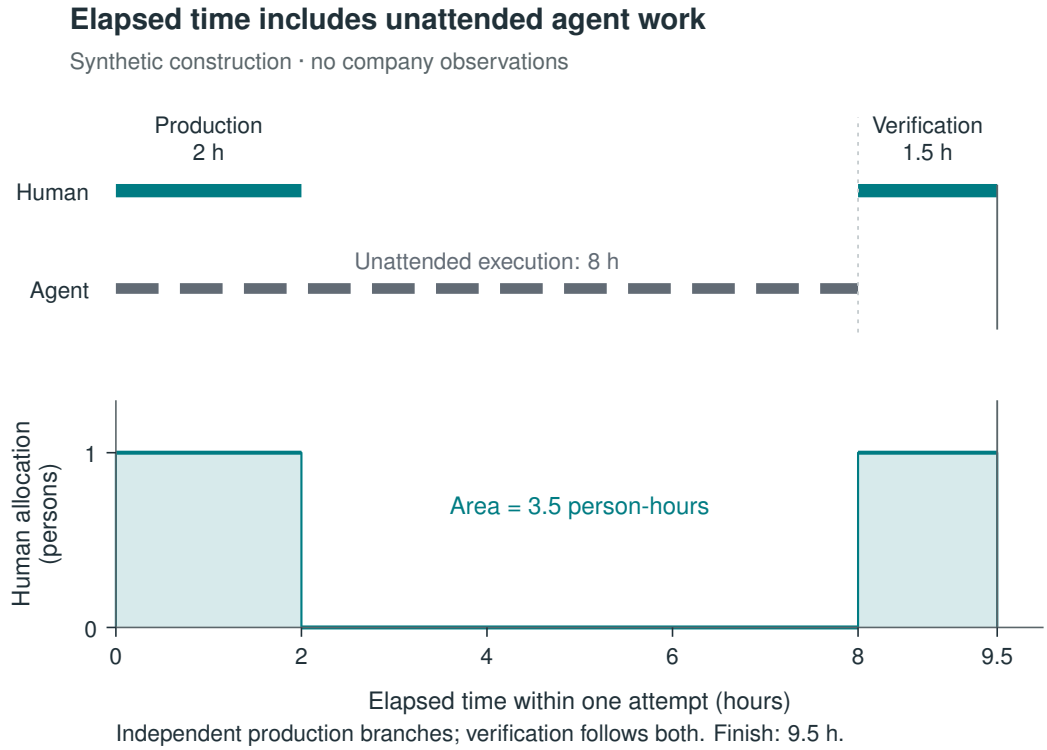


FIGURE 1. One synthetic attempt with $c_h = 2$, $g = 8$ and $v = 1.5$ hours. Human production and agent execution begin together, and verification waits for both. Human labour is 3.5 person-hours, whereas elapsed duration is 9.5 hours. The figure concerns one attempt and excludes the initial queue and subsequent retries.

4.2. **First acceptance under repeated attempts.** Assume independent, identically distributed pass/fail attempts with pass probability $p \in (0, 1]$, deterministic per-attempt labour $h > 0$ and duration $t > 0$. Attempts execute sequentially until the first pass; N is their number. A one-off initial wait is $w \geq 0$. Write $H^A = hN$ for labour to first acceptance and $L = w + tN$ for elapsed time. These quantities exclude later repair.

Proposition 4.2 (Retry moments and cycle rates)**.** *Under these assumptions,*

$$\mathbb{E}[N] = \frac{1}{p}, \quad \mathbb{E}[H^A] = \frac{h}{p}, \quad \mathbb{E}[L] = w + \frac{t}{p}, \quad \mathrm{Var}(H^A) = \frac{h^2(1-p)}{p^2}. \tag{12}$$

Suppose independent, identically distributed task cycles have fixed score $S^0 \geq 0$, bounded retained fraction R with $\mathbb{E}[R] = r$, and the same parameters above. Ratios of accumulated retained output to accumulated pre-acceptance labour and to serial cycle duration converge almost surely to

$$P_H^A = \frac{S^0 r p}{h}, \qquad V_{\text{serial}} = \frac{S^0 r}{w + t/p}. \tag{13}$$

Dependence of R on the attempt count within a cycle is allowed.

Proof. The tail probability is $\mathbb{P}(N > j) = (1-p)^j$ for integers $j \geq 0$, whose sum is $1/p$. For $p < 1$, conditioning on the first outcome gives $\mathbb{E}[N^2] = 1 + (1-p)(2\mathbb{E}[N] + \mathbb{E}[N^2])$, so $\mathbb{E}[N^2] = (2-p)/p^2$. The moments are finite because the geometric tail decays; hence $\mathrm{Var}(N) = (1-p)/p^2$. At $p = 1$, $N = 1$ surely. Scaling gives (12). For the rates, apply the

strong law separately to the iid vectors (S^0R, H^A, L) and divide by the positive limiting mean denominators. This yields the ratios of expectations in (13). □

These are elementary renewal-reward calculations [7]. They are ratios of totals, not expectations of task-specific ratios. The serial rate describes one repeated stream; it is not a formula for the throughput of concurrent teams. The observation of R can occur after completion, provided all cycles are eventually assessed on the same basis. A wait repeated on every attempt belongs inside the multiplier $1/p$.

The eventual first-acceptance model also differs from (7) through its deadline: every modelled task eventually passes, whereas the cohort numerator credits only acceptance by τ.

Example 4.3 (A horizon changes the ratio)**.** Set $p = 1/2$, $h = t = S^0 = R = 1$, $w = 0$ and $\tau = \Delta = 1$, with one person working continuously on sequential attempts until acceptance and no repair. Then $\mathbb{E}[Q] = 1/2$ and $\mathbb{E}[H(2)] = \mathbb{E}[\min(N, 2)] = 3/2$, so their ratio is $1/3$, while eventual-cycle productivity is $1/2$. These expectations follow from the probabilities of first-attempt success and second-attempt execution, each $1/2$.

The retry formula is a sensitivity model, not a plug-in formula for a finite cohort. A calendar deadline also need not equal an attempt cap when durations or waits vary.

The first-acceptance labour boundary is deliberately narrower than (7). If $B_\Delta \geq 0$ is attributable labour during a specified post-acceptance follow-up with finite expectation, then the corresponding cycle productivity has denominator $h/p + \mathbb{E}[B_\Delta]$. That extension does not capture all labour in a fixed enrolment horizon automatically. Neither quantity includes unallocated shared overhead unless the measurement protocol adds it.

Corollary 4.4 (Separate improvement conditions)**.** *For policies $a \in \{0, 1\}$ satisfying the retry assumptions, expected labour and expected intrinsic duration decrease respectively if and only if*

$$\frac{h_1}{h_0} < \frac{p_1}{p_0}, \qquad \frac{t_1}{t_0} < \frac{p_1}{p_0}. \tag{14}$$

Here intrinsic duration excludes the initial wait. If $S^0 > 0$ and $r_0 > 0$, policy 1 improves P_H^A if and only if

$$\frac{h_1}{h_0} < \frac{r_1 p_1}{r_0 p_0}. \tag{15}$$

Proof. Substitute (12) and (13), then multiply by the strictly positive denominators. The positivity of S^0 and r_0 is required for the last equivalence. □

Equal initial waits preserve the intrinsic-duration comparison. With unequal waits, compare $w_1 + t_1/p_1$ and $w_0 + t_0/p_0$ directly. A smaller drafting cost establishes none of these inequalities on its own.

Example 4.5 (A retry reversal)**.** Changing (h, p) from $(1, 0.9)$ to $(0.5, 0.2)$ halves per-attempt labour but raises expected labour from $10/9$ to $5/2$ person-hours by (12). Lower effort per attempt can be overwhelmed by additional attempts.

4.3. **Stopping, heterogeneity and the limits of the mechanism.**

Proposition 4.6 (A finite attempt cap)**.** *Under independent attempts with $p \in (0, 1]$, allow at most $K \geq 1$ attempts and count exhaustion without a pass as failure. The probability q_K of acceptance and expected attempt count m_K satisfy*

$$q_K = 1 - (1 - p)^K, \qquad m_K = \mathbb{E}[\min(N, K)] = \sum_{j=0}^{K-1} (1 - p)^j = \frac{q_K}{p}. \tag{16}$$

Let $r_K = \mathbb{E}[R \mid N \leq K]$ be the conditional mean retained fraction for acceptance within the cap. Expected labour is hm_K and expected retained output is $S^0 r_K q_K$.

Proof. Acceptance is the complement of K consecutive failures. The finite tail-sum identity gives the expectation, and multiplication of the geometric sum by $p = 1 - (1 - p)$ telescopes. The output formula follows by conditioning on acceptance; it assigns zero output to exhaustion while retaining its labour. □

At $p = 0$, the direct boundary definition gives $m_K = K$ and $q_K = 0$. At $K = 0$, both totals vanish and their ratio is undefined. Whenever $q_K = 0$, expected retained output is zero directly; the conditional quantity r_K is not evaluated. For $p > 0$ and $K \geq 1$, the ratio of expected retained output to labour is $S^0 r_K p/h$. Identically distributed accepted-quality marks independent of earlier attempts would give $r_K = r$ and a ratio independent of the positive cap. Without that extra condition, r_K may vary. Even when cancellation holds, the cap still changes the chance of delivering an individual request.

Example 4.7 (Retention depends on the cap)**.** Let $p = 1/2$ and $R = \mathbf{1}\{N = 1\}$. Then $r = \mathbb{E}[R] = 1/2$ but $r_1 = 1$. At $K = 1$, true expected retained output is $S^0/2$; substituting the uncapped r would incorrectly give $S^0/4$. Thus iid pass indicators alone do not make retained quality independent of the retry history.

Remark 4.8 (Task heterogeneity). If per-attempt labour h is fixed and a task-specific pass probability P lies in $(0, 1]$, then, whenever $\mathbb{E}[1/P]$ is finite,

$$h\mathbb{E}[1/P] \geq h/\mathbb{E}[P]. \tag{17}$$

This follows from convexity of $x \mapsto 1/x$ and Jensen's inequality. Replacing heterogeneous probabilities by their mean can therefore understate effort. The conclusion does not extend unchanged to arbitrary dependence between effort and pass probability.

Real attempts often learn from earlier failures and can change both cost and acceptance probability. A retry model then needs history-dependent conditional probabilities, or direct totals that avoid that model. Shared queues, changing reviewer availability and abandonment also violate the elementary cycle description. These are measurable departures to investigate, not reasons to force every task into a geometric distribution.

5. Acceptance, correctness and synthetic checks

An acceptance gate is part of the observation process. More passing work can mean better production, easier assessment or both. To separate these mechanisms, suppose an attempt is defective with probability $d \in [0, 1)$ and the gate detects a defect with sensitivity $s \in [0, 1]$. Correct attempts are never rejected. Attempts are independent with the same composition, and the gate stops at the first pass.

Proposition 5.1 (Detection changes the meaning of throughput)**.** *The pass probability and conditional correctness are*

$$p = 1 - ds, \qquad \rho = \mathbb{P}(\textit{correct} \mid \textit{pass}) = \frac{1 - d}{1 - ds}. \tag{18}$$

For fixed $S^0 > 0$ and $t > 0$, no initial wait and no repair cost, the serial recorded and correct-output rates are

$$V_{\text{recorded}} = \frac{S^0(1 - ds)}{t}, \qquad V_{\text{correct}} = \frac{S^0\rho(1 - ds)}{t} = \frac{S^0(1 - d)}{t}. \tag{19}$$

For $0 < d < 1$, lowering s strictly raises the recorded rate and strictly lowers correctness among accepted attempts.

Proof. Rejection requires both a defect and detection, so its probability is ds. Every correct attempt passes, and Bayes' rule gives (18). Summing over the number of earlier rejected attempts preserves the pass-conditioned composition at first acceptance. Equation (13) with $w = 0$, first with reward fraction 1 and then with ρ, gives (19). For positive d, lowering s increases the positive denominator of ρ while its numerator remains positive and fixed. □

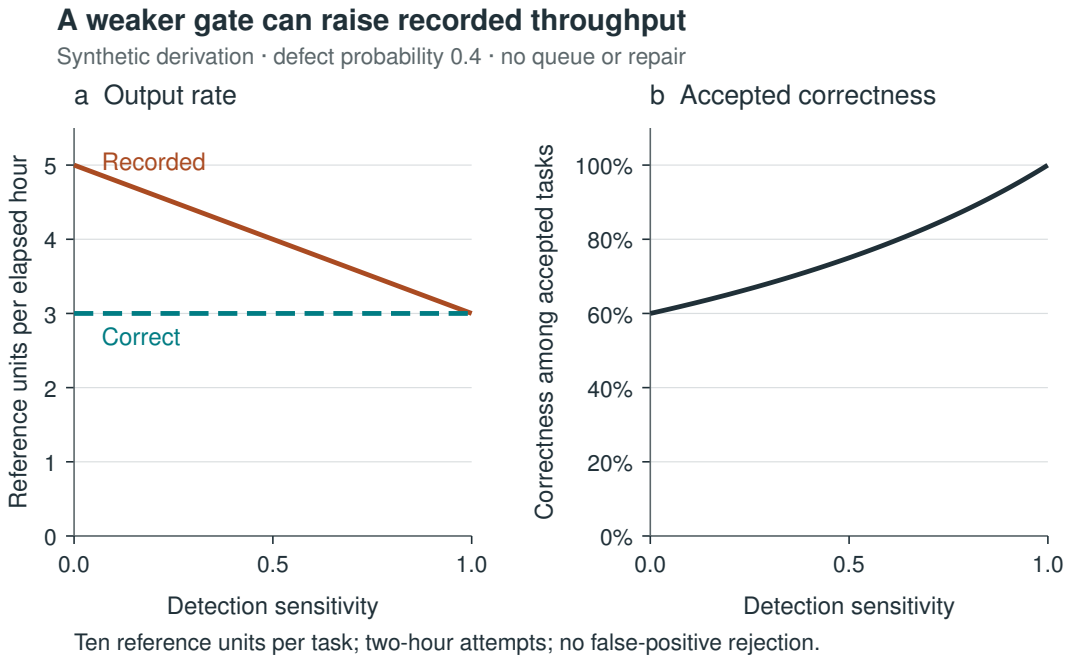


FIGURE 2. Consequences of the independent-attempt detection model for the synthetic settings $d = 0.4$, $t = 2$ hours and $S^0 = 10$ reference units. Weaker detection raises recorded throughput and lowers correctness among accepted attempts. Correct-output throughput is constant only under the stated model, which omits escaped-defect consequences and repair. Conditional correctness ρ is distinct from the general follow-up fraction R.

TABLE 1. Synthetic first-acceptance results from (11)–(13). Human labour is measured in person-hours; elapsed time includes the initial wait. Productivity excludes repair.

Workflow	$\mathbb{E}[H^A]$	$\mathbb{E}[L]$	P_H^A
Human baseline	7.37	9.37	1.330
AI: shorter production	3.89	5.89	2.520
AI: slower agent branch	3.89	12.56	2.520
AI: greater verification	10.67	12.67	0.844

The cancellation in correct-output rate is a property of this particular mechanism. It gives no recommendation to weaken testing. Escaped defects may impose repair costs, affect other tasks or violate an essential quality constraint. At $d = 0$ there is no detection effect. With false-positive rejection, heterogeneous attempts or nonconstant costs, the formula changes. Observed gate failures alone cannot separate defect prevalence from detection sensitivity; an independent assessment sample is needed.

Table 1 and Figure 3 illustrate the independent effort and time conditions. Every task has $S^0 = 10$, one-off waiting time $w = 2$ hours and no post-acceptance repair. The four parameter tuples (c_h, g, v, p, r) are $(6, 0, 1, .95, .98)$, $(2, 1, 1.5, .90, .98)$, $(2, 8, 1.5, .90, .98)$ and $(1, .5, 7, .75, .90)$. These are chosen inputs, not estimates from developer data.

The shorter-production and slower-agent cases consume the same expected human labour, but only the former arrives sooner than the baseline. In the greater-verification case, reduced human production is outweighed by review and retries. The examples demonstrate logically possible mechanisms; they do not establish how often those mechanisms occur.

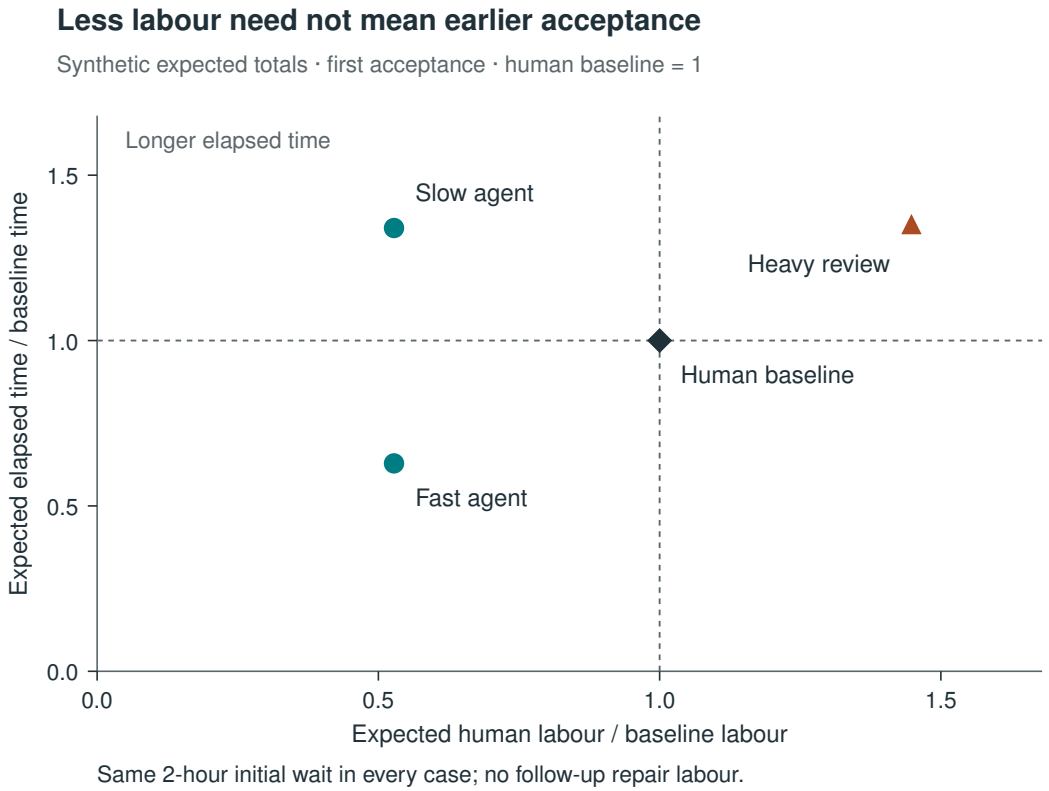


FIGURE 3. Synthetic expected human labour and elapsed time relative to the human baseline. The two equal-labour AI cases differ only in agent duration. They demonstrate that lower human effort does not imply faster acceptance. The plotted values are exact model expectations, with decimal rendering; they are not effect estimates or uncertainty intervals.

The executable supplement checks the rational arithmetic, counterexamples and finite-cap boundaries. Its Monte Carlo experiment uses seeds 17, 41 and 93 and 100,000 independent task draws per scenario per seed. It samples geometric attempt counts and human labour; elapsed-time and retained-output values in Table 1 are analytic. From (12), for the mean $\overline{H}_n$ of n independent task-labour draws,

$$\sigma_n^2 = \operatorname{Var}(\overline{H}_n) = \frac{h^2(1-p)}{np^2}, \qquad \mathbb{P}(|\overline{H}_n - h/p| \le 10\sigma_n) \ge 0.99. \tag{20}$$

The probability bound follows from Chebyshev's inequality for $p < 1$ and exact equality of the mean for $p = 1$. This is not a simultaneous guarantee across seeds or scenarios. Agreement checks the implementation under its generating assumptions, not the empirical truth of those assumptions.

The mathematical proofs, finite computations and empirical questions have different scopes. Appendix B records the precise formal coverage. No stochastic simulation tests a causal policy effect, and no algebraic proof establishes that diaries, acceptance gates or retained-output scores are reliable in an organisation.

6. A STUDY THAT COULD TEST THE MEASUREMENT PROPOSAL

The immediate empirical objective is to test whether the proposed observations can be collected reliably and whether a frozen change index is calibrated to labour under each policy. Estimating an AI effect is a subsequent inferential objective whose precision depends

TABLE 2. Proposed measurements and the ambiguities each resolves. Collection details are to be registered before the pilot.

Record	Minimum content	Purpose
Intake	Requirement, scope version, enrolment, eligibility, frozen score	Preserve the unit and pre-policy task mix
Labour	Role, task allocation, interval, sampling probability, nonresponse	Estimate all-role person-hours without double counting
Lifecycle	Start, attempts, review, acceptance, deployment, abandonment	Separate waiting, execution and unsuccessful work
Exposure	Assigned policy, tools, versions, actual use, resource charges	Distinguish access, use and nonhuman cost
Follow-up	Common rubric, blinded audit, defects, repair and assessment date	Separate gate acceptance from retained functionality

on the available independent teams, task variation and measurement error. A small pilot can settle feasibility without pretending to settle effectiveness.

6.1. **A bounded prospective protocol.** One implementable starting protocol recruits 12 teams with substantially distinct review queues, matched into six pairs using a two-week run-in. Matching uses repository maturity, task classes and baseline workload, all recorded before assignment. Within each pair, randomly assign one team to the specified AI policy and one to the comparator. Enrol new tasks over six weeks. These numbers are proposed logistics, not a power calculation. If review or agent capacity is shared extensively, the assignment unit must include that shared pool; otherwise the proposed team randomisation does not isolate the intervention.

Use a predeclared class of maintenance and small feature tasks with a common intake process. Lock a baseline backlog before assignment and report it separately from later arrivals. Register each baseline task's enrolment relative to assignment and the treatment of any pre-assignment work, without silently resetting its clock or excluding effort. For new arrivals, apply the same eligibility and scope rules before implementers select tasks or expose them to the tools. Record all eligible requests, including those postponed or abandoned. This distinguishes an effect on execution of a fixed workload from a policy-induced change in which work is undertaken. Expanding scope is an outcome to report, not a reason to relabel a larger task as the original one.

For the initial pilot, take $\tau = 30$ calendar days from enrolment and $\Delta = 30$ days after acceptance. Maintain the assigned policies and observation through the final required follow-up, 60 days after the last enrolment. The values may be unsuitable for other work classes and must be changed before that cohort begins. Document model versions, permitted autonomous actions, prompt and review practices, outages and comparator-tool access. Avoid a short crossover design: learning, code changes and queue effects may persist after a switch.

The minimal data record has five linked components, shown in Table 2. Task identifiers and participant roles connect the records. Identifiers used for analysis should be pseudonymous, with access to detailed work traces limited to the research purpose. The protocol need not capture private thought or treat keyboard activity as continuous labour.

For labour, a workable run-in samples four short intervals per participant-day at random from the declared work frame and asks for role and task allocation for each sampled interval. Retain the inclusion probabilities and response timestamps. Independently adjudicate a random subset using diaries and task records, without using the policy label where feasible. Measure response burden, coverage of after-hours work and disagreement. Sampling intensity can be adjusted using the run-in variance and burden data before the trial protocol is frozen.

Persistent missingness is reported by policy, role and task class; expanding telemetry cannot by itself repair unobserved reasoning time.

For quality, assess a probability sample of accepted requirements at acceptance and after Δ using the same tests and review rubric in both groups. Reviewers should be blinded to policy where the artefact permits it, and failures of blinding should be recorded. Duplicate assessment of a subset permits reporting agreement, disagreement patterns and adjudication changes. Retention scores need known assessment probabilities when extrapolated to the full cohort. Prespecify when findings are fed back and count audit-triggered repair within the applicable labour boundary. If assessments change subsequent work, define policy effects under a common assessment and feedback regime. Extrapolation from the audited subset then requires a design and assumptions accounting for audit exposure; known assessment probabilities alone are insufficient. Preserve severity-specific defects and actual repair labour as separate outcomes: a single retained fraction cannot encode every consequence of a security or reliability failure.

6.2. **Estimands that retain unsuccessful tasks.** The primary arm-level summaries are mean all-role cohort labour, acceptance by τ, restricted time without acceptance, mean retained output and their aggregate productivity ratio. For potential outcomes under assigned policy $a \in \{0, 1\}$, define

$$\begin{aligned} \mu_H(a) &= \mathbb{E}[H_i^a(\tau + \Delta)], \quad F_A^a(u) = \mathbb{P}(T_i^a \le u), \\ \mu_Q(a) &= \mathbb{E}[Q_i^a(\tau, \Delta)], \qquad\quad P_H^a = \mu_Q(a)/\mu_H(a), \end{aligned} \tag{21}$$

with a positive labour mean for the last ratio. For the locked baseline backlog, the expectation weights each eligible task equally; functional size enters only through S_i^0 in Q_i. Later arrivals require a separate estimand because the policy can change which requests arrive. A team-level adoption estimand instead uses a common calendar window and the full eligible portfolio, including changes in intake and work in progress, as in (8); it must not condition on treatment-induced submissions. The population and weighting must therefore accompany every contrast. Differences between the policy means, and between the two policy-level ratios, target assignment rather than selective use. A reduction in labour is not a benefit by itself if it results from abandoning more requests; the other endpoints reveal that possibility.

Proposition 6.1 (Restricted time without acceptance)**.** *For any nonnegative extended acceptance time T, including $T = \infty$, and finite $\tau > 0$,*

$$M_A(\tau) = \mathbb{E}[\min(T, \tau)] = \int_0^\tau [1 - F_A(u)]\, du, \qquad F_A(u) = \mathbb{P}(T \le u). \tag{22}$$

Proof. For each outcome, $\min(T, \tau) = \int_0^\tau \mathbf{1}\{T > u\}\, du$. Tonelli's theorem permits exchanging expectation and the nonnegative integral. □

Smaller M_A means less time spent without acceptance during the specified horizon. Always report $F_A(\tau)$ alongside it. With complete follow-up to τ, empirical averages estimate both directly. Permanent abandonment remains nonacceptance in this definition; it is not dropped as a missing completion time. If abandonment is modelled as a terminal competing event, estimate acceptance cumulative incidence. Treating abandonment as independent censoring instead changes the target.

Administrative censoring before τ requires additional assumptions. Unconditional independent censoring permits ordinary survival estimation when there is no terminal competing event and observation support extends through the target horizon. Conditional independence requires the corresponding support within relevant at-risk covariate strata, together with conditioning or appropriate weighting before marginal standardisation. Support means a positive probability of remaining observed wherever unaccepted tasks contribute to the target, including its endpoint. If support ends earlier, shorten the target horizon or state explicit extrapolation assumptions or bounds. Informative loss to follow-up requires a sensitivity

analysis or bounds. A task that has not yet reached its follow-up date is not evidence of zero repair or perfect retention. The planned full follow-up reduces these problems but does not guarantee that losses will be absent.

6.3. **Analysis, measurement reliability and a decision.** Analyse all eligible tasks under their assigned policy. Unequal team backlog sizes require care: a difference of pooled arm task means need not be design-unbiased for the fixed task-weighted contrast. For a locked backlog of known size $n_0 > 0$, let $\mathcal{I}_g$ be the fixed task set of team g, A_g its assignment, and $\pi_g(a) = \mathbb{P}(A_g = a) = 1/2$. For an observed task outcome Y_i, use the assignment-weighted mean

$$\widehat{\mu}_Y(a) = \frac{1}{n_0} \sum_g \frac{\mathbf{1}\{A_g = a\}}{\pi_g(a)} \sum_{i \in \mathcal{I}_g} Y_i. \tag{23}$$

Under consistency and the stated absence of cross-team interference, its assignment expectation is the finite-backlog mean of Y_i^a: replace each observed summand on $A_g = a$ by its potential outcome, then use $\mathbb{E}[\mathbf{1}\{A_g = a\}/\pi_g(a)] = 1$. The same argument applies to the policy-mean difference. Labour and quality subsampling additionally require their own design weights and measurement assumptions. The estimate $\widehat{\mu}_Q(a)/\widehat{\mu}_H(a)$ remains a ratio estimator, not an unbiased ratio; report it only with a positive denominator.

Respect the matched-pair assignment and retain numerator–denominator dependence and assignment clusters in inference; thousands of commits do not create thousands of independent treatment assignments. With only six pairs, randomisation-based comparisons are coarse and intervals may be wide. Report feasibility and measurement results even if an effect estimate is inconclusive. Use run-in and pilot estimates of clustering, outcome dispersion, attrition and assessment error to design a larger trial around a prespecified precision or power target. No claim of adequate power follows from the illustrative recruitment count.

The observation model has its own validation targets. Compare sampled labour with frozen-index predictions by policy, role and task class. Examine calibration in hours, absolute error, interval coverage and systematic residual shifts on held-out teams or periods. High correlation of binned means is insufficient. Record whether task splitting, squashing or representation changes alter the score materially. For quality, report inter-reviewer disagreement and follow-up completeness, alongside the outcome estimates. Sensitivity analyses should vary plausible missing labour and disputed retention scores; if a policy ranking reverses, the measurement uncertainty is consequential.

Start prediction with direct accounting and a small task-class baseline. More elaborate models should earn their complexity through held-out performance. Features must exist at the declared prediction time. A model for the mean of log labour does not directly predict arithmetic mean labour after exponentiation; retransformation depends on the residual distribution. Zero-human-effort tasks need an explicit treatment, and marginal effort and time predictions do not determine the probability of meeting both a budget and a deadline without their dependence. Even a randomised comparison of arm means does not identify the cross-policy probability that the same task would improve under both unobserved counterfactuals. These qualifications matter before a forecasting model is used for staffing.

Randomisation can support a causal policy contrast only for the implemented assignment and its interference assumptions. Shared reviewers, reusable AI-generated components and changes to colleagues' queues create spillovers. Cluster at a boundary that contains the main shared resources, record cross-boundary dependencies and describe residual interference. If randomisation is unavailable, a staggered adoption design may use the cohort-specific estimands of Callaway and Sant'Anna [3], subject to its comparison-group, no-anticipation, overlap and conditional parallel-trends assumptions. A step in SCH must not itself define adoption timing. Reversible or changing use is not automatically a persistent adoption treatment.

Before examining outcomes, define the practical labour or timing improvement sought and a domain-justified quality margin. Report both intention-to-treat endpoint contrasts and their uncertainty. Failure to detect a quality reduction is not evidence that a noninferiority margin is met. A policy can be supported for labour saving while leaving delivery speed unresolved, or can create an explicit trade-off. Retained-output productivity is informative only with its numerator and denominator also reported. A favourable index movement alone is insufficient for a staffing, quality or delivery claim.

7. Computing repeated cohort summaries

The accounting definition in (7) also determines a small computational problem. An analyst may compare many acceptance horizons on the same observed cohort while keeping the follow-up duration Δ fixed. Repeatedly scanning every record gives a simple reference implementation. Sorting the records once and storing cumulative totals replaces each subsequent scan with two binary searches. This changes the cost of computing the reported quantities; it does not change the quantities or establish an effect of AI assistance on human labour.

7.1. **A discrete input contract.** Let $A = ((a_i, w_i))_{i=1}^{n}$ contain one accepted-output record per accepted requirement, with acceptance age a_i measured from that requirement's enrolment and retained weight w_i assessed at the common follow-up Δ. Let $B = ((b_j, v_j))_{j=1}^{m}$ contain task-attributable labour contributions, including contributions from unfinished, rejected and abandoned tasks. Ages use a declared common integer time unit. Output weights use millionths of a reference unit; labour weights use millionths of a person-hour. All ages and weights are nonnegative `u64` integers. The two weight types are summed separately.

For an integer horizon τ, the computational targets are

$$Q_{\mathrm{d}}(\tau) = \sum_{i=1}^{n} w_i \mathbf{1}\{a_i \le \tau\}, \qquad H_{\mathrm{d}}(\tau, \Delta) = \sum_{j=1}^{m} v_j \mathbf{1}\{b_j \le \tau + \Delta\}. \tag{24}$$

The result is the exact integer pair $(Q_{\mathrm{d}}, H_{\mathrm{d}})$. Their common scale cancels in the productivity ratio, which is defined only when $H_{\mathrm{d}} > 0$. A zero denominator remains an explicit zero in the pair; the Rust convenience method `ratio()` returns `None` in that case. Its floating-point result when the denominator is positive is a display approximation, not the canonical stored value.

This contract does not discretise the continuous labour integral automatically. A contribution assigned to the end of a sampled interval or time bin approximates an interior cutoff. Equality with (4) requires an observation and integration rule that supplies the correct contributions for the queried boundaries, for example cutoffs aligned with already integrated bins. Missing labour, unobserved retention, duplicated tasks and scope changes must be resolved by the measurement protocol; the index cannot diagnose them. Retained weights are valid for the particular Δ at which they were assessed. Changing Δ, the cohort or its assessed weights requires corresponding input data and a rebuilt index.

7.2. **Construction, queries and correctness.** The sorted prefix-index constructor, Algorithm 1, is a deterministic preprocessing algorithm for the two fixed record streams. The order of records with equal ages is immaterial: every such record is included or excluded together. Arrays use one-based record positions and a zero prefix. CheckedAdd128 widens the weight and either returns the exact sum or reports `TotalOverflow`; no wrapped or partial index is returned. The Rust implementation copies the input streams before sorting, so construction preserves the caller's records.

The batch horizon evaluator, Algorithm 2, is a read-only query algorithm on this index. UpperBound(K, x) returns the number of sorted keys at most x, including every duplicate at the boundary. CheckedAdd64 reports `HorizonOverflow` if $\tau + \Delta$ cannot be represented.

Algorithm 1 Build exact cumulative totals for a fixed cohort and follow-up

Require: Accepted-output records A and labour records B under the stated contract.
Ensure: Sorted keys and exact prefix totals, or `TotalOverflow`.

for all stream names S in the ordered list (A, B) **do**
  $X_S \leftarrow \textsc{SortCopyByAge}(S)$
  $K_S \leftarrow$ age array of X_S; $P_S[0] \leftarrow 0$
  for $j \leftarrow 1$ **to** $|X_S|$ **do**
    $z \leftarrow \textsc{CheckedAdd128}(P_S[j-1], X_S[j].\text{weight})$
    if $z =$ overflow **then**
      return `TotalOverflow`
    end if
    $P_S[j] \leftarrow z$
  end for
end for
return $I = (K_A, P_A, K_B, P_B)$

Algorithm 2 Return exact output and labour totals for a batch of horizons

Require: Index I; horizons $\tau_1, \ldots, \tau_q$; the fixed follow-up Δ.
Ensure: Ordered exact total pairs, or `HorizonOverflow`; no division by zero.

for $r \leftarrow 1$ **to** q **do**
  $c_r \leftarrow \textsc{CheckedAdd64}(\tau_r, \Delta)$
  if $c_r =$ overflow **then**
    return `HorizonOverflow`
  end if
end for
$O \leftarrow$ empty result array
for $r \leftarrow 1$ **to** q **do**
  $k_A \leftarrow \textsc{UpperBound}(K_A, \tau_r)$
  $k_B \leftarrow \textsc{UpperBound}(K_B, c_r)$
  Append $(P_A[k_A], P_B[k_B])$ to O
end for
return O

The entire batch is checked before any result is returned. Query order is preserved, so horizons need not arrive sorted.

Proposition 7.1 (Agreement with a full scan)**.** *If index construction and horizon addition succeed, each returned pair in Algorithm 2 equals* (24)*. This includes empty streams, repeated ages, zero weights and zero labour totals.*

Proof. After construction has processed j records of a sorted stream, its loop invariant is $P_S[j] = \sum_{\ell=1}^{j} X_S[\ell].\text{weight}$. It holds at $j = 0$ and checked addition preserves it. Sorting preserves the multiset of records. For a cutoff x, upper-bound binary search maintains a lower prefix whose keys are at most x and an upper suffix whose keys exceed x. Its remaining interval strictly decreases until its bounds coincide at k. Thus exactly positions $1, \ldots, k$ meet the inclusive cutoff. The corresponding prefix total is the same sum a full scan selects. Apply this argument at $x = \tau$ for output and $x = \tau + \Delta$ for labour. The empty prefix has total zero, so the boundary cases follow without division. □

The proof concerns these discrete sums and the published operations. It is an analytical proof, not an extension of the Lean proof inventory in Appendix B. The Rust tests check implementation agreement with a separate scanning path; successful tests do not establish completeness or reliability of the underlying observations.

TABLE 3. Measured cohort-query times in milliseconds: medians of 15 paired observations at each query count. Indexed total is timed separately and includes index disposal; component medians need not sum to it. Reduction is $100(1 - \text{indexed median/scan median})$, so a negative value denotes a regression.

q	Scan	Build	Query	Indexed total	Reduction
1	2.889	20.348	0.002	21.016	-627.4%
10	28.003	20.206	0.008	20.709	26.0%
100	263.415	22.348	0.082	23.046	91.3%
1000	2787.159	21.803	0.508	23.051	99.2%

With n acceptance records, m labour records and q queries, repeated scanning takes $O(q(1+n+m))$ time. Construction takes $O(n\log(n+1)+m\log(m+1))$ time and $O(n+m)$ additional storage. Indexed queries take $O(q(1+\log(n+1)+\log(m+1)))$ time and $O(q)$ result storage. These bounds count fixed-width operations and assume available memory. Preprocessing can outweigh the saving for a small batch, and an index cannot be reused after its input changes without maintaining or rebuilding its prefix totals.

7.3. **A reproducible performance comparison.** The supplement's dependency-free Rust program compares the scanning and indexed implementations on the same generated records and horizon batches. It retains the exact totals for equality checks and times computation separately from data generation. The benchmark reports index construction as well as query time: a fast query alone does not establish that building the index was worthwhile. Its fixed follow-up and synthetic inputs isolate this implementation choice; they do not represent a company workload or estimate a policy effect. For further reading on data science with Rust, see Van Der Post [11]; the runtime claims here rely on the accompanying measurements.

The workload contains 100,000 generated accepted-output records and 400,000 generated labour increments; it has no task roster or task-to-increment associations. Ages and horizons use hours, with $\Delta = 720$ and queried horizons from 0 to 1,440. Three fixed seeds (17, 41 and 93) define the datasets and horizon sequences. For each seed and each $q \in \{1, 10, 100, 1000\}$, five fresh processes each run one paired comparison. Each process first performs an unmeasured full scan followed by an unmeasured indexed run; measured method order then alternates across observations. These are timings on warmed data and code, not cold-start measurements. Both implementations return identical integer pairs in every warmup and measured comparison. All 60 measured pairs are retained.

The build uses Rust 1.85.1, optimisation level 3, one code-generation unit, no link-time optimisation and enabled overflow checks, on a shared Linux host reporting an Intel Xeon Platinum 8573C processor. Dataset generation, startup, warmup, output verification and result I/O are outside both timed regions. Indexed total time includes input copies, sorting, prefix construction, queries and disposal of the index. The environment record retains the compiler details, source hashes and timing boundaries. Host load is uncontrolled; the 15 pairs per query count are repeated measurements of three synthetic datasets, not independent samples of production work.

Table 3 retains the adverse small-batch result. For one query, indexing increases median elapsed time from 2.889 to 21.016 ms. At ten queries, the median decreases by 26.0%, but individual paired reductions range from −16.4% to 55.1%: indexing does not win every observed comparison. At 100 and 1,000 queries, every measured pair favours the index. For 1,000 queries, the median falls from 2,787.159 to 23.051 ms, a 99.2% reduction including construction and disposal. Figure 4 shows the observed ranges as well as the medians. The ranges describe variation across seeds and repeated host runs; they are not confidence intervals. The tested query counts do not identify an exact break-even point.

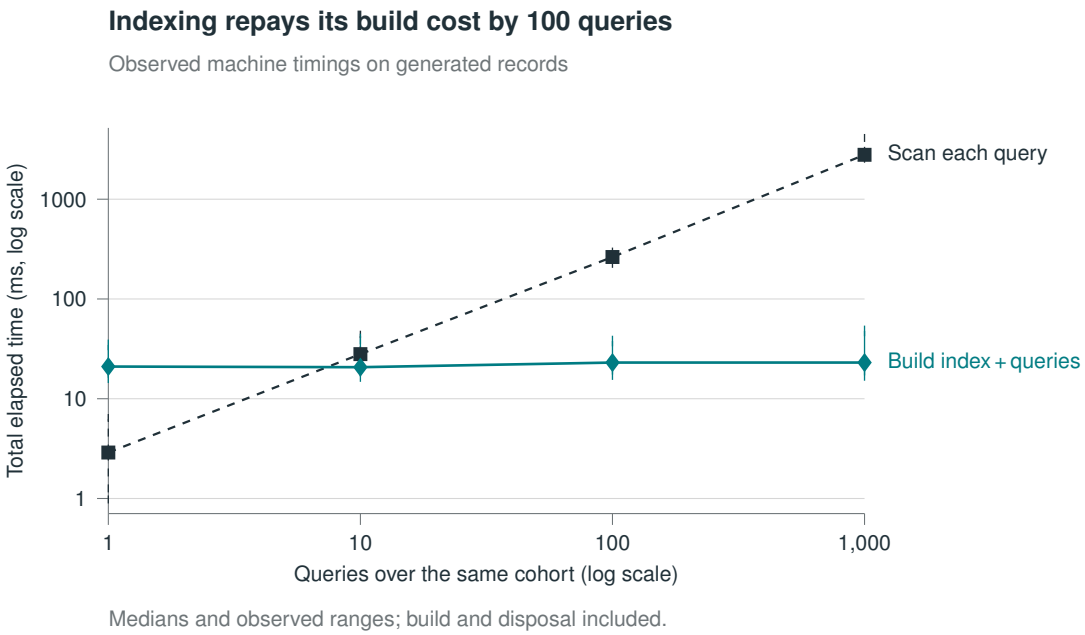


FIGURE 4. Observed elapsed times for repeated cohort-horizon queries on generated arrays of 100,000 accepted-output events and 400,000 labour increments. Markers are medians; whiskers span 15 paired runs (three seeds, five fresh-process repetitions per seed), not confidence intervals. Indexed totals include construction, queries and disposal. All one-query pairs favoured scanning; all 100-query pairs favoured indexing. Ten-query pairs varied in direction. Only four query counts were timed; connecting lines do not locate break-even. This result is confined to the generated workload and execution environment and does not estimate human labour or an AI-policy effect.

The observed saving is consistent with amortising preprocessing over repeated scans. It supports the index for this fixed workload and query pattern, subject to its storage and update costs. It does not establish performance on larger-than-memory datasets, continuously changing ledgers, different hardware or different query distributions. Raw paired timings, the generator, exact-output checks and the summary script are retained so that another workload can test those boundaries. Faster analysis of an unchanged ledger remains separate from lower measured human effort, earlier task acceptance or better retained output.

8. Scope and remaining work

The strongest established result is a measurement limitation: event and change observations do not supply human effort, elapsed delivery and retained output without explicit assumptions or additional observations. The counterexample establishes this limitation within an admissible event-model subfamily. The execution models establish possible mechanisms and exact conditional comparisons. They do not determine which mechanism dominates in practice.

The proposed empirical contribution remains to be tested. It may prove too costly to collect reliable all-role allocations, difficult to freeze comparable tasks or impossible to blind quality assessment. A frozen reference can drift as task composition and engineering practices change. The treatment may also alter the requirements requested, so a fixed-task experiment and an organisation-wide adoption study answer different questions. These are substantive limits, not gaps filled by more precise algebra.

The next research step is the measurement pilot: establish the observation frame, task identity, allocation reliability, follow-up completeness and feasibility of independent assignment. Only then can an organisation calibrate the reference scale, estimate a policy effect or justify a forecasting model. The common reference score remains a potentially useful index, with the resource and output quantities required to interpret it made explicit.

Appendix A. Targeted corrections to the Standard Coder argument

The following corrections concern identifiable statements in the seven-page version of [12]. They do not constitute reproduction of the fitted models or an exhaustive audit. Raw observations, model artefacts and split manifests were not supplied.

Proposition A.1 (The stated sign-test rejection set)**.** *For* $K \sim \mathrm{Binomial}(10, 1/2)$*,*

$$\mathbb{P}(K \in \{0,1,9,10\}) = \frac{\binom{10}{0} + \binom{10}{1} + \binom{10}{9} + \binom{10}{10}}{2^{10}} = \frac{11}{512} = 0.021484375. \tag{25}$$

Proof. The four disjoint event masses have numerators $1, 10, 10, 1$, whose sum is 22. Divide by 1024. □

The stated probability in Section III.B of the source is 5.52%, whereas its stated rejection set gives approximately 2.15% under independent fair signs. A second binomial calculation for 24 positives among 300 tests gives an upper tail of approximately 5.42×10^{-8} with the corrected probability. This is conditional arithmetic, not an empirical rejection result: fair signs and independence within and across tests require justification. A mean-zero correction need not have equally probable signs. Neither calculation establishes calibration of hidden activity states.

Example A.2 (A correlation does not isolate efficiency variation)**.** Let X and Z be independent and uniform on $\{-1, 1\}$. Set $S = 2 + X$, $Y = 10 + 4X + 3Z$ and $U = 3X + 3Z$, so that $Y = 10 + X + U$. Then

$$\mathrm{Corr}(S, Y)^2 = \frac{16}{25}, \qquad \frac{\mathrm{Var}(U)}{\mathrm{Var}(Y)} = \frac{18}{25} > 1 - \frac{16}{25}. \tag{26}$$

Proof. The independent centred signs have unit variance. Thus $\mathrm{Var}(S) = 1$, $\mathrm{Var}(Y) = 25$, $\mathrm{Cov}(S, Y) = 4$ and $\mathrm{Var}(U) = 18$, which give the displayed values. Both S and Y are strictly positive. □

Consequently, the residual fraction $1 - r^2$ cannot in general upper-bound the variance of a designated efficiency component. Such a bound needs a structural decomposition and appropriate orthogonality assumptions; project averages do not themselves bound individual variation. The example supplies a logical counterexample, not a causal allocation of variance in data. Correlation also fails to establish calibration: multiplying an outcome by two leaves its positive correlation with itself equal to one.

Two reporting issues remain separate from these deductions. Figure 6's caption reverses the timing and code-feature provenance of target and prediction. Its approximate bin counts also imply about 1.2 million examples, compared with about 500,000 commits in the dataset description. An alternative counting unit or a typographical error could reconcile the latter; the unavailable manifest is needed to decide. Neither issue proves that the underlying observations or implementation are invalid. Feature substitutions in a fitted predictor are sensitivity calculations and need a separate identification argument before being interpreted causally.

Appendix B. Proof coverage and reproducibility

The Lean 4.33.1 project retained with the 13 October 2026 manuscript, pinned to mathlib v4.33.1, contains 43 named theorems, including supporting lemmas. Its retained receipt records a successful source build and audit of every exported theorem's axioms. The original manuscript, proof sources and receipt remain together in the unchanged baseline package. Section 7, its algorithms and its Rust implementation are outside that receipt's coverage; they have separate analytical arguments and executable tests, not new Lean proofs. The declarations use only the recorded standard Lean axioms `propext`, `Classical.choice` and `Quot.sound`, where required; no admitted proof, custom axiom or native-decision shortcut is used. The receipt records each declaration's actual assumptions and source hash. These checks establish the Lean statements, with the coverage boundaries below, rather than every assertion in the article.

Article claim	Checked content and remaining boundary
Proposition A.1	The actual ten-sign space has 1,024 elements; the rejection event has 22. Its uniform probability is 11/512. Fairness and independence of empirical signs remain assumptions.
Proposition 2.1	Explicit hidden-state marginalisation gives equality for every finite binary string at the two rational parameter pairs, and their finite-history expected activity differs for every positive length. The infinite-process construction and interpretation as labour remain analytical.
Example A.2	The actual four-point means, covariances, variances, positivity and decomposition are checked. The designated component has no established causal interpretation.
Proposition 4.2	Convergent geometric weighted sums establish total mass, first and second moments, central second moment, scaled labour variance and waited duration. The iid retry interpretation and the strong-law limit of cycle ratios remain analytical.
Proposition 4.6	A finite weighted tail identity, its geometric-tail specialisation and closed form are checked, including $p = 0$, $K = 0$ and $p = 1$ boundaries. Acceptance and retention assumptions are not inferred from the identity.
Corollary 4.4 and Proposition 5.1	Positive-denominator equivalences, a normalised nonnegative defect/gate table, conditional correctness, rate cancellation and strict detection trade-offs are checked. First-pass composition and empirical adequacy require the stated probabilistic argument.
Propositions 3.1 and 4.1	Finite inverse-probability weighting and the capacity-bound algebra are checked. Measurement accuracy, the schedule-to-capacity premises, the DAG recursion and the causal interpretation of (23) remain separate.

The `Basic`, `Binomial` and `Geometric` modules contain the proofs. The complete inventory and semantic map in the supplement distinguish these statements from deferred conditional-expectation, strong-law, Jensen and restricted-time integral proofs. An analytical proof is not relabelled as machine-checked merely because a related arithmetic lemma compiles. The Python calculations are tested implementations, not formally proved refinements of the Lean definitions.

The original supplement retains the earlier generic numerical regression benchmark for provenance. It is not a validation of the task model, the empirical study or causal inference, and it supplies no performance result used in the article. Reproducibility requires the source, fixed synthetic inputs, seed record, software environment and build logs. A successful computation or proof check establishes only the claim encoded by that artefact.

The 13 October 2026 baseline was developed through a communicating multi-agent workflow using MAS Math Lab 0.3.0, its existing Rust/nom structured-message codec, and separate drafting, mathematical, source-audit and checking contexts. The earlier delivered exchange was retained as an input. The protocol was not newly invented or adapted for this article. Agent reviews and Lean checks are disclosed development aids, not external

peer review; the 2019 source paper is the work of its cited authors. The retained package preserves the correction records and execution limits; the complete development message archive is not included. Empirical validation and specialist review remain necessary before submission.

References

1. Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein, *Measuring the impact of early-2025 AI on experienced open-source developer productivity*, 2025, arXiv:2507.09089v2. https://arxiv.org/abs/2507.09089v2.
2. Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud, *We are changing our developer productivity experiment design*, METR research update, February 2026, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/.
3. Brantly Callaway and Pedro H. C. Sant'Anna, *Difference-in-differences with multiple time periods*, Journal of Econometrics **225** (2021), no. 2, 200–230, https://arxiv.org/abs/1803.09015v4.
4. Kevin Zheyuan Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, June 2025, Author manuscript; Section 4.1 and Table 3. https://www.mertdemirer.com/Papers/Demirer_AI_productivity.pdf.
5. ———, *The effects of generative AI on high-skilled work: Evidence from three field experiments with software developers*, Management Science (2026), Published online 27 February 2026. https://doi.org/10.1287/mnsc.2025.00535.
6. Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler, *The SPACE of developer productivity: There's more to it than you think*, ACM Queue **19** (2021), no. 1, 20–48, https://doi.org/10.1145/3454122.3454124.
7. Robert Gallager, *Lecture 12: Renewal rewards, stopping trials, and Wald's equality*, 6.262 Discrete Stochastic Processes, MIT OpenCourseWare, 2011, Lecture slides, 14 March; renewal rewards on slide 8. https://ocw.mit.edu/courses/6-262-discrete-stochastic-processes-spring-2011/resources/mit6_262s11_lec12/.
8. D. G. Horvitz and D. J. Thompson, *A generalization of sampling without replacement from a finite universe*, Journal of the American Statistical Association **47** (1952), no. 260, 663–685, https://doi.org/10.1080/01621459.1952.10483446.
9. Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz, *Reading between the lines: Modeling user behavior and costs in AI-assisted programming*, Proceedings of the CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, 2024, Author manuscript: arXiv:2210.14306v5. https://arxiv.org/abs/2210.14306v5, pp. 142:1–142:16.
10. Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer, *The impact of AI on developer productivity: Evidence from GitHub Copilot*, 2023, arXiv:2302.06590v1. https://arxiv.org/abs/2302.06590v1.
11. Hayden Van Der Post, *Data Science with Rust: The Speed & Safety Revolution*, Reactive Publishing, 2024, Available via Reactive Publishing.
12. Ian Wright and Albert Ziegler, *The standard coder: a machine learning approach to measuring the effort required to produce source code change*, 2019, arXiv:1903.02436v1. https://arxiv.org/abs/1903.02436v1.